Data Analytics Project: Scottish Hills Dataset Analysis

Dataset Used: scottish_hills.csv

Team Members: Subbaraju | Sumanth | Sruthi | Swetha | Tarun|

This document outlines a structured, end-to-end data analysis project based on the Scottish Hills dataset. The project is divided into five scenarios, increasing in difficulty from basic data handling to advanced analysis and visualization, ensuring comprehensive skill coverage.

Project Goal: To analyse Scottish hill heights, regional distributions, and elevation patterns using the scottish_hills.csv dataset.

The primary libraries/modules required for this project are:

- **Pandas:** For all data manipulation, filtering, aggregation, and feature engineering tasks.
- **NumPy:** Specifically for numerical calculations like height differences (np.diff).
- **Visualization Library (e.g., Matplotlib):** For generating all required plots (Line Graphs, Bar Charts, Pie Charts, Stacked Bar Charts, Histograms).

Project Coverage:

Category	Skills Covered
Data Pre-processing	Data Loading, Missing Value Handling (Mean/Mode), Type Conversion
Pandas Operations	Grouping, Filtering, Aggregation, Feature Engineering (Height Category)
NumPy Calculations	Array conversion, Height difference calculation (np.diff)
Visualization	Line Chart, Bar Chart, Pie Chart, Histogram, Stacked Bar Chart, Saving Plots

Scenario 1: Data Loading & Basic Cleaning

Description:

In this scenario we are loading the data set to the pandas DataFrame and displaying the top 5 rows. After that we are checking for any missing values and if found any, we need to replace the height column with mean and region column with mode of those columns respectively. And we need to convert the type of height column to numeric if necessary.

Task	Description
1.	Load the Scottish Hills dataset using Pandas.
2.	Display the first 5 rows (head ()) and column names.
3.	Check for missing values in Height and Region.
4.	Handle missing values: Fill Height with the mean, Region with the mode.
5.	Convert the Height column to numeric if required.

Output:

The file is loaded to pandas DataFrame. The top 5 rows are displayed. We have checked for any missing values. The height column is converted to numeric type. And the scenario 1 result is as shown below.

```
                    Hill Name  Height  ...    Osgrid      Region
0       A' Bhuidheanach Bheag   936.0  ...  NN660775  South-East
1                A' Chailleach   997.0  ...  NH136714  North-West
2                A' Chailleach   929.2  ...  NH681041  North-East
3  A' Chraileag (A' Chralaig)  1120.0  ...  NH094147  North-West
4               A' Ghlas-bheinn   918.0  ...  NH008231  North-West

[5 rows x 6 columns]
Missing value count:
Height    0
Region    0
dtype: int64
Before converting Data type of Height column is: float64
After converting Data type of Height column is: float64
```

Conclusion:

In this scenario we loaded the csv file to pandas DataFrame. And displayed the first 5 rows along with column name. There are no missing values in the height and region columns. The code has been written to add mean and mode to the NaN cells. And also the height column is converted to numeric type as requested.

Scenario 2: Line Graph + Save (Easy)

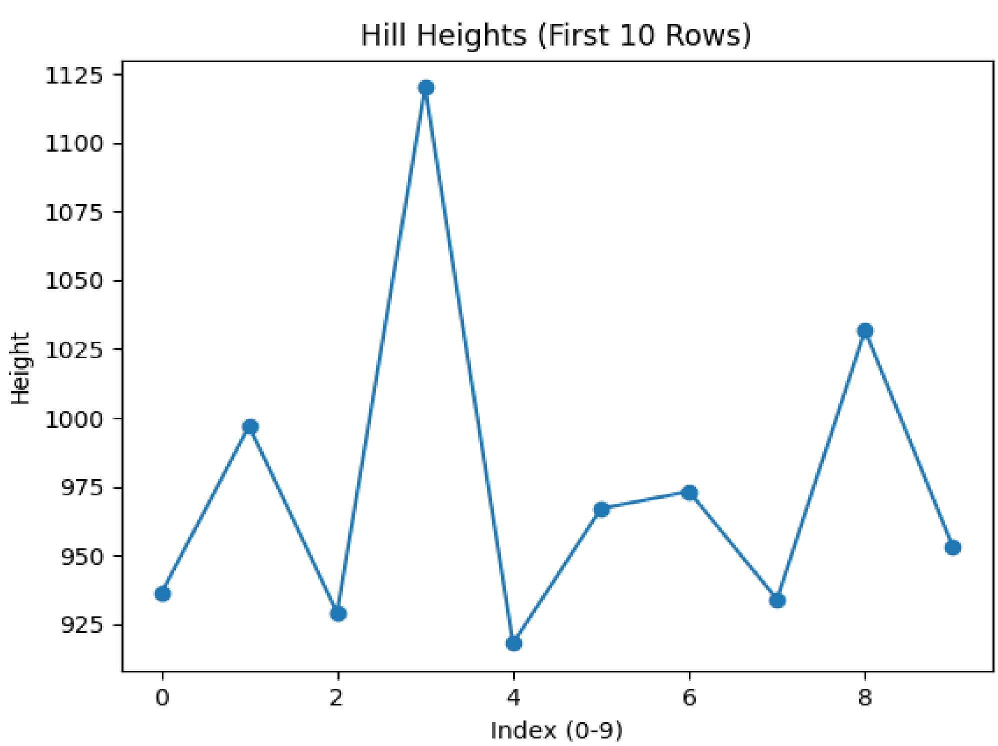
Description:

This scenario focuses on analysing how hill heights vary. The Hill Name and Height columns are selected from the dataset using Pandas. Only the first 10 rows are taken for easy visualization. The Height values are converted into a NumPy array using NumPy. A line graph is plotted using Matplotlib to show how heights change across the selected hills.

Task	Description
1.	Select Hill Name and Height columns.
2.	Take first 10 rows only.
3.	Convert Height into a NumPy array.
4.	Plot a line graph: X-axis → index (0-9), Y-axis → Height.
5.	Add title and axis labels.
6.	Save the graph: plt.savefig("hill_heights_line.png")

Graph:

The graph is a line chart where the X-axis represents the index values (0–9) and the Y-axis represents the height of the hills. Each point on the graph shows the height of a hill, and the points are connected by a line. The graph helps in visualizing how the heights increase or decrease across the selected hills.



Conclusion

The line graph shows variation in hill heights among the selected data. Some hills are higher while others are lower. The graph helps in easily comparing the heights. Overall, it gives a clear understanding of how hill heights vary.

Scenario 3: Filtering + Bar Chart + Save (Medium)

Description:

This scenario focuses on filtering and visualising tall hills from the dataset. Hills with Height > 900 are extracted to focus on significant elevations. The filtered data is grouped by Region and the count of tall hills per region is calculated. Top regions are selected, converted to NumPy arrays, and visualised as a bar chart saved as tall_hills_bar.png.

Task	Description
1.	Filter hills where Height > 900.
2.	Count the number of hills per Region.
3.	Select top regions.
4.	Convert results to NumPy arrays.
5.	Plot a bar chart: X-axis → Region, Y-axis → count.
6.	Rotate x-axis labels for clarity.
7.	Save the graph: plt.savefig("tall_hills_bar.png")

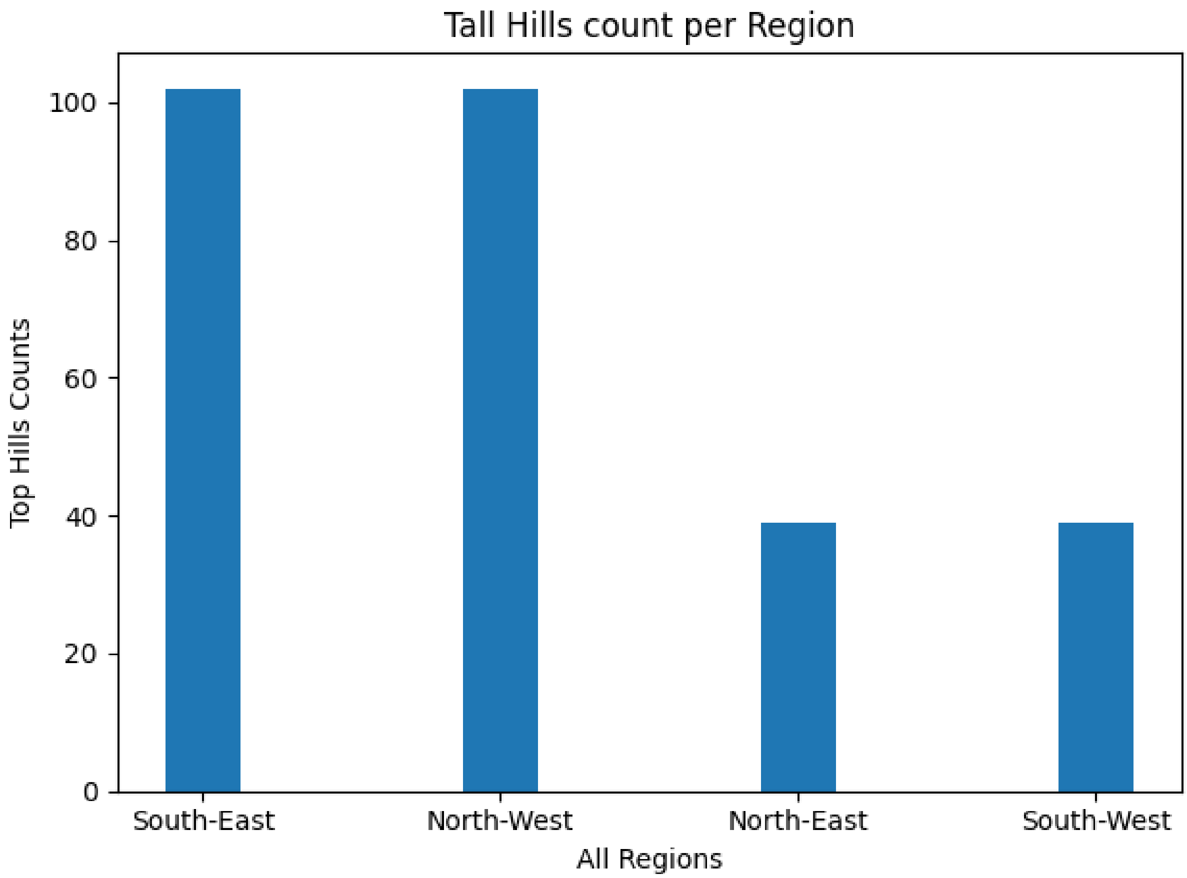
Output:

In this scenario we are counting the total number of hill stations per region. And filtering the dataset to get the regions with tallest hill counts and plotting the bar chart to provide the graphical representation. The regions with highest counts is as shown below.

```
Region
South-East     102
North-West     102
North-East      39
South-West      39
Name: count, dtype: int64
Regions with top Hill counts: ['South-East', 'North-West']
['South-East' 'North-West'] <class 'numpy.ndarray'>
```

Graph:

From the Graph we can conclude that the South-East and North-West regions have the most number of tallest hills as shown below.



Conclusion:

In this scenario we filtered the dataset with the regions having tallest hil stations. And we calculated the total number of hill-stations per region. Also we created the top regions to numpy array and we have represented the result graphically using bar chart. We can conclude that South-East and North-West regions have the most number of tallest hills stations.

Scenario 4: Pie Chart (Region Distribution) + Save (Medium)

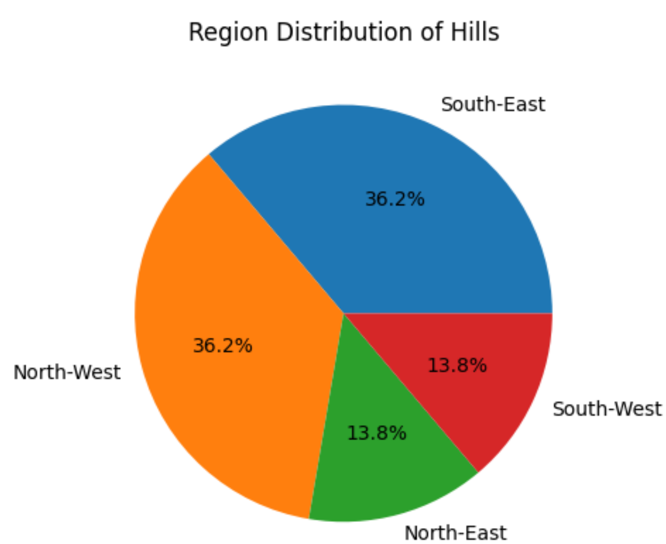
Description:

This scenario focuses on analyzing the distribution of hills across different regions in Scotland. The goal is to understand how hills are spread geographically by calculating the number of hills present in each region. The top 5 regions with the highest number of hills are selected for better visualization. A pie chart is then used to represent the proportion of hills in each region, making it easy to compare regional distribution visually. Percentage labels are added to clearly show each region's contribution to the dataset.

Task	Description
1.	Count the number of hills per Region.
2.	Select top 5 regions.
3.	Prepare labels and values for the pie chart.
4.	Plot a pie chart using Region as labels and count as values.
5.	Add percentage labels using autopct (e.g., '%.1f%%').
6.	Save the graph: plt.savefig("region_distribution.png")

Graph:

The pie chart visually represents the proportion of hills in the top 5 Scottish regions. Each slice corresponds to a region, and the size of the slice indicates its share of the total hills. The percentage labels help in clearly identifying the contribution of each region. The Highlands region occupies the largest portion, showing it has the highest number of hills compared to other regions.



Conclusion:

The analysis clearly shows that the Highlands region dominates in terms of hill distribution, while other regions such as Cairngorms, Lochaber, Grampian, and Perthshire have smaller shares. This visualization effectively highlights the uneven geographical distribution of hills in Scotland and helps identify the most hill-dense regions.

Scenario 5: Advanced Analysis + Multiple Graphs (Hard)

Description:

This advanced scenario performs multi-dimensional analysis on the Scottish Hills dataset by combining data transformation, NumPy computations, and multiple visualisations. A new Height Category column classifies hills as Very High, High, or Moderate. NumPy's np.diff() calculates height differences across the dataset.

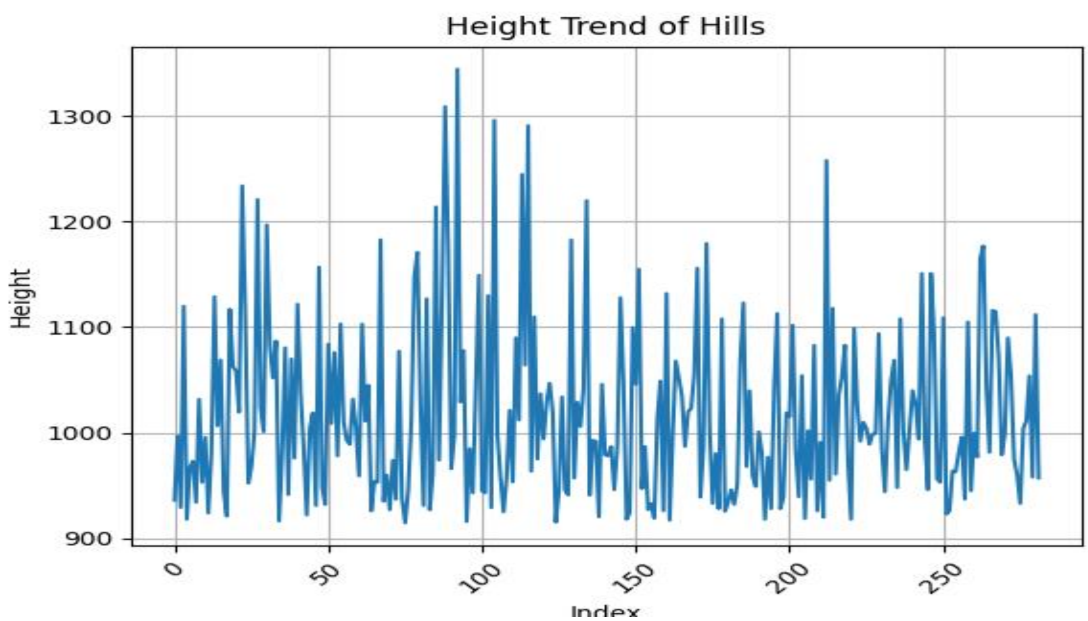
Task	Description
1.	Create Height Category column: Height >= 1000 → "Very High", 800-999 → "High", < 800 → "Moderate".
2.	Convert Height column to NumPy array.
3.	Calculate height differences using np.diff().
4.	Plot a Line Graph showing height trend for all hills.
5.	Plot a Stacked Bar Chart: count of Height Category per Region.
6.	Plot a Histogram: distribution of Height.
7.	Save graphs: plt.savefig("height_trend.png"), plt.savefig("height_category_stacked.png"), plt.savefig("height_histogram.png")
8.	Identify: Which region has the tallest hills, which category is most common, and the distribution pattern of heights.

Graphs:

Three graphs been plotted. Line graph, Stacked bar graph and Histogram graph for better visualization.

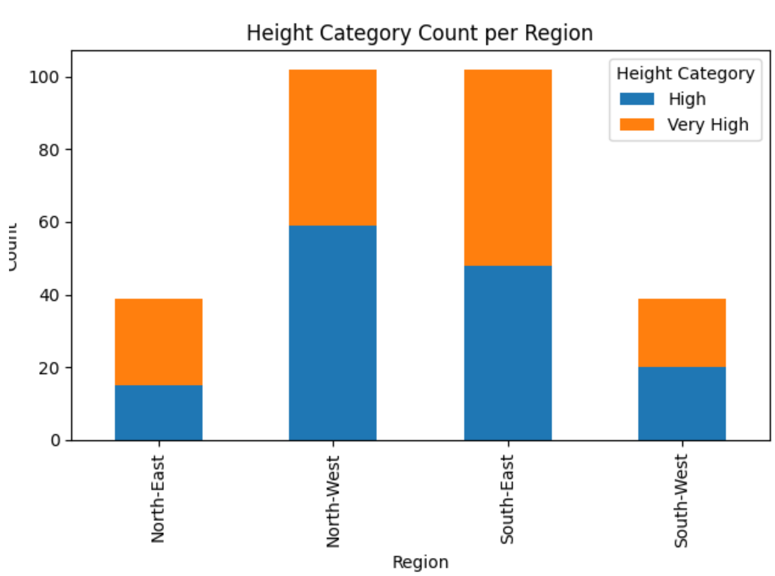
Line Graph

This line graph shows how the heights of hills vary across the dataset based on their index position. The values fluctuate between lower and higher heights, indicating no consistent upward or downward trend It helps visualize the spread and randomness of hill heights in the dataset.



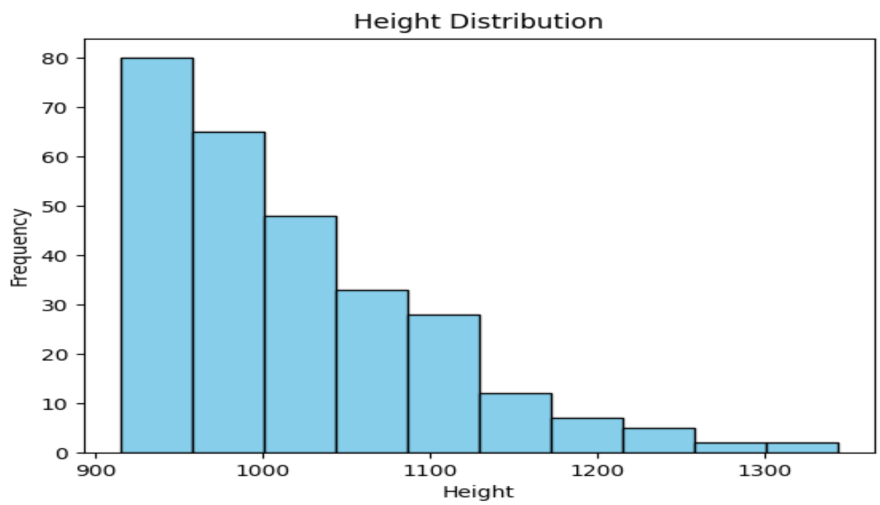
Stacked Bar Chart

The stacked bar chart shows the count of each Height Category per Region. The Highlands region has the highest proportion of Very High and High category hills. Moderate category hills are more evenly distributed across regions.



Histogram

The histogram reveals the distribution of hill heights across the dataset. The distribution shows a slight right skew, with most hills falling in the 700-1000m range and fewer hills exceeding 1100m, consistent with Scotland's Munro and Corbett classification system.



Conclusion:

From the above data set we have plotted all the graphs that are required for us to understand the data in visuals. We have plotted line Graph showing height trend for all hills. The stacked bar chart shows the count of each Height Category per Region and the histogram reveals the distribution of hill heights across the dataset.

Final Conclusion

This project analysed the Scottish Hills dataset using Pandas, NumPy, and Matplotlib. It showed that the Highlands region has the highest number of tall hills, and most hills fall in the 800–1000m range. The data visualisations helped clearly understand patterns in hill height and regional distribution. Overall, the analysis gave useful insights into Scotland’s landscape.

What We Have Learned

In this project, we have learnt how to clean and process data using Pandas and perform simple calculations using NumPy. We have also learnt how to create different visualisations like line graphs, bar charts, pie charts, and histograms using Matplotlib. Most importantly, we understood how to turn raw data into meaningful insights using an end-to-end data analysis process.